TECHNICAL REPORT
Book Metadata and Recommendation API

---

1. Introduction

This report describes the design and implementation of the Book Metadata and Recommendation API developed for the COMP3011 Web Services and Web Data coursework.

The goal of the project was to develop a RESTful API capable of managing a dataset of book metadata while demonstrating correct usage of HTTP endpoints, JSON responses, and database integration.

The system allows users to create, retrieve, update, and delete book records. In addition to CRUD operations, the API provides analytical endpoints that allow users to explore patterns in the dataset, such as genre trends and rating distributions.

---

2. System Overview

The API provides a backend service for managing book metadata. The system consists of three main components:

1. A FastAPI application that exposes RESTful endpoints.
2. An SQLite database used to store book records.
3. A CSV dataset that is imported into the database on first run.

Users interact with the API through HTTP requests. The API processes these requests, performs the appropriate database operations, and returns responses in JSON format.

---

3. Technology Stack

Python

Python was used as the primary programming language due to its simplicity and strong ecosystem for web development.

FastAPI

FastAPI was chosen as the framework because it provides high performance, automatic validation, and built-in API documentation through Swagger UI.

SQLite

SQLite was used as the database backend because it is lightweight and does not require a separate database server. This makes it well suited for small applications and coursework projects.

Uvicorn

Uvicorn is an ASGI server used to run the FastAPI application locally.

---

4. Dataset

The API uses a CSV dataset containing metadata about books.

The dataset includes information such as:

• Book titles
• Authors
• Genres
• Ratings
• Publication details
• Publisher information
• Pricing data

When the application is first executed, the dataset is automatically imported into an SQLite database.

Using a database rather than reading the CSV file directly allows the API to support full CRUD operations and efficient querying.

---

5. Database Design

The application uses a single table called "books".

Each record in the table represents a book and contains metadata fields imported from the dataset.

An auto-incrementing field called id is used as the primary key. This field is used by the API when performing CRUD operations.

The original dataset identifier, bookId, is stored as a normal attribute rather than being used as the primary key.

---

6. API Design

The API follows a RESTful architecture where resources are accessed using HTTP methods.

The main endpoints support CRUD operations:

GET /books – retrieve books
GET /books/{id} – retrieve a specific book
POST /books – create a new book
PUT /books/{id} – update a book
DELETE /books/{id} – delete a book

Additional endpoints provide analytical insights into the dataset, including genre trends and rating distributions.

---

7. Validation and Error Handling

The API uses Pydantic models to validate request bodies.

Two models are defined:

BookCreate – used when creating a new book
BookUpdate – used when updating an existing book

These models ensure that incoming requests contain correctly formatted data.

The API also returns appropriate HTTP status codes to indicate the result of each request.

---

8. Testing

The API was tested using the interactive documentation automatically generated by FastAPI.

Testing included creating new book records, retrieving books from the database, updating existing records, and deleting records.

Error handling was also tested by requesting non-existent book IDs and sending invalid request bodies.

---

9. Limitations

The current implementation has several limitations:

• No authentication mechanism has been implemented.
• Genres and authors are stored as plain text rather than normalized relational tables.
• Recommendation functionality is limited to simple analytics queries.

---

10. Future Improvements

Several improvements could be made in future versions of the API:

• Implement authentication for write operations
• Add filtering and search functionality
• Normalize the database schema into multiple related tables
• Implement more advanced recommendation algorithms
• Deploy the API to a cloud hosting platform

---

11. Conclusion

The Book Metadata and Recommendation API successfully demonstrates the implementation of a RESTful web service using FastAPI and SQLite.

The project provides CRUD functionality, analytics endpoints, and automatic API documentation. It demonstrates key concepts in web services development including RESTful design, request validation, and database integration.

---

12. Generative AI Declaration

Generative AI tools were used during the development of this project to assist with debugging, code explanation, and documentation drafting.

All generated content was reviewed, modified, and verified by the author before inclusion in the final submission.

---

13. Appendix – Chat Logs

(a) Minimum Technical Requirements

To achieve a Pass (i.e. 40+), your API must at a minimum:

• implement at least one data model involving all four elements of CRUD (Create, Read, Update and Delete) functionality linked to a database.2 You can base your design on a RESTful (Representational State Transfer) API. However, you can also use a GraphQL-based API if you think this is more suitable for the problem, but you must explain the rationale for using GraphQL.

• includes at least four API endpoints accessible via HTTP requests

• handle user inputs and return appropriate JSON responses. You are permitted to use other formats, provided you justify these in your supporting report document.

• use correct status/error codes, according to industry convention

• be demonstrable via local execution, web hosting (e.g. PythonAnywhere), or a Model Context Protocol (MCP) server.

You can base your implementation on Django, but you can also use

other suitable stacks (e.g. Python FastAPI, Node.js Express, Go

Fiber). You must justify your technical choices and motivations.

(b) Example Project Ideas

• A book metadata and recommendation API integrating a public

dataset (e.g. Goodreads or Google Books) and providing analytic

endpoints such as genre trends, rating distributions, and person-

alised suggestions.

---+

A solid way to satisfy this brief is to build a **Book Metadata and Recommendation API** with one core database-backed model that fully supports CRUD, plus a few analytics endpoints.

Recommended project concept

Project title: Book Metadata and Recommendation API

What it does:

Stores books in your own database, lets users create/read/update/delete them, and adds extra endpoints for recommendations and analytics.

This fits the brief well because it gives you:

- one clear CRUD model: Book
- at least four HTTP endpoints
- JSON request/response handling
- meaningful status codes
- an easy demo locally or on hosting

Suggested stack

I would recommend:

- **FastAPI** for the API
- **SQLAlchemy** or **SQLModel** for database access
- **SQLite** for development
- **Uvicorn** to run locally

Why this is a strong choice:

- FastAPI is lightweight and very suitable for REST APIs
- automatic interactive docs at /docs
- built-in validation via Pydantic
- easy JSON handling
- easy to demonstrate

Django is also acceptable, but FastAPI is often easier for a focused API assignment.

Core data model

Use one main model first:

Book

Example fields:

- id
- title
- author
- genre
- published_year
- rating
- description

That is enough to implement all four CRUD operations.

You could later add optional models like:

- User
- Review
- Favorite

But for the pass requirement, one proper CRUD model is enough.

CRUD endpoints

These four already meet the minimum requirement:

- POST /books → create a book
- GET /books → list all books
- GET /books/{id} → get one book
- PUT /books/{id} → update a book
- DELETE /books/{id} → delete a book

That is actually five endpoints, which is even better.

Extra endpoints for higher quality

To make the project more interesting and closer to the example brief, add analytics/recommendation endpoints such as:

- GET /books/recommendations?genre=fantasy&min_rating=4.0
- GET /analytics/genre-trends
- GET /analytics/rating-distribution

Examples:

GET /books/recommendations

Returns books matching user preferences such as genre or minimum rating.

GET /analytics/genre-trends

Returns counts of books by genre.

GET /analytics/rating-distribution

Returns how many books fall into rating bands.

These make the API look more thoughtful than basic CRUD alone.

Example JSON

Create book request

```
{
  "title": "The Hobbit",
  "author": "J.R.R. Tolkien",
  "genre": "Fantasy",
  "published_year": 1937,
  "rating": 4.8,
  "description": "A fantasy adventure novel."
}
```

Success response

```
{
  "id": 1,
  "title": "The Hobbit",
  "author": "J.R.R. Tolkien",
  "genre": "Fantasy",
  "published_year": 1937,
  "rating": 4.8,
  "description": "A fantasy adventure novel."
}
```

Status codes to use

Use proper industry-standard codes:

- 200 OK → successful GET/PUT
- 201 Created → successful POST
- 204 No Content → successful DELETE
- 400 Bad Request → invalid input
- 404 Not Found → book does not exist
- 422 Unprocessable Entity → validation errors in FastAPI
- 500 Internal Server Error → unexpected server failure

That directly addresses the brief.

Database requirement

The brief says the CRUD functionality must be **linked to a database**.

So make sure your books are stored in SQLite, not just an in-memory Python list.

Example:

- development DB: books.db
- table: books

This is important for meeting the technical requirement.

Demonstration plan

You can demonstrate it by running locally:

```
uvicorn main:app --reload
```

Then test via:

- browser docs at http://127.0.0.1:8000/docs
- Postman
- curl

This satisfies the “demonstrable via local execution” part.

Good report justification

In your report, you can justify the design like this:

Why REST instead of GraphQL

Choose REST because:

- the assignment only needs straightforward CRUD and analytics endpoints
- REST is simpler to implement and test
- HTTP methods map naturally to CRUD operations
- it is widely used and easy to explain

Only choose GraphQL if your API needs highly flexible nested queries from clients. For this project, REST is the simpler and stronger choice.

Why FastAPI

- fast to build
- automatic validation
- automatic OpenAPI docs
- easy JSON serialization
- well suited for academic API demos

Why SQLite

- lightweight
- no setup burden
- enough for a coursework prototype
- integrates well with Python

Simple marking-safe structure

A good structure would be:

```
project/
│
├── main.py
├── models.py
├── schemas.py
├── database.py
├── crud.py
├── requirements.txt
└── README.md
```

Minimum viable version

If you want the simplest version that still safely meets the pass criteria, build:

- Book model in SQLite
- POST /books
- GET /books
- GET /books/{id}
- PUT /books/{id}
- DELETE /books/{id}
- one extra analytics endpoint like GET /analytics/genre-trends

That would be enough for a credible submission.

Stronger version

To push above a basic pass, add:

- filtering: GET /books?genre=Fantasy
- sorting: GET /books?sort_by=rating
- recommendation endpoint
- input validation
- seeded sample dataset
- Swagger docs screenshots in report
- error handling examples in report

Best recommendation

For this assignment, I would build:

A RESTful Book Metadata and Recommendation API using FastAPI + SQLite, with:

- full CRUD for books
- analytics endpoints for genres and ratings
- JSON input/output
- proper HTTP status codes
- local demo through FastAPI docs

That is realistic, easy to justify, and closely aligned with the brief.

I can also draft the full FastAPI codebase for this project.

---+

ERROR: Error loading ASGI app. Attribute "app" not found in module "main".

---+

That error means Uvicorn found your main.py file, but **inside that file there is no variable named app**.

When you run:

```
uvicorn main:app --reload
```

Uvicorn reads it as:

- main = the Python module main.py
- app = an object inside that file called app

So your file must contain something like:

```
from fastapi import FastAPI

app = FastAPI()
```

Most likely fix

Make sure your main.py looks roughly like this:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def root():
    return {"message": "Hello World"}
```

Then run:

```
uvicorn main:app --reload
```

Common reasons this happens

1. You named it something other than

app

For example, if your code says:

```
api = FastAPI()
```

then this command will fail:

```
uvicorn main:app --reload
```

because there is no app.

You would either change the code to:

```
app = FastAPI()
```

or run:

```
uvicorn main:api --reload
```

---

2. Your file is not actually called

main.py

If your file is called server.py, then use:

```
uvicorn server:app --reload
```

Format is always:

```
uvicorn filename_without_.py:fastapi_variable_name --reload
```

---

3. Your

app = FastAPI()

is inside a function or

if __name__ == "__main__":

Bad example:

```
from fastapi import FastAPI

if __name__ == "__main__":
    app = FastAPI()
```

Uvicorn usually won't see that correctly.

It should be at top level:

```
from fastapi import FastAPI

app = FastAPI()
```

---

4. There is another error in

main.py

Sometimes Uvicorn reports the app is missing when the file failed to import properly.

For example:

```
from fastapi import FastAPI
import something_that_does_not_exist

app = FastAPI()
```

Then the module may fail before app is created.

Try running:

```
python main.py
```

If there is another import error, it should show up there.

Working minimal example

Create a file called main.py with this exact code:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "API is working"}
```

Then start it with:

```
uvicorn main:app --reload
```

Then open:

```
http://127.0.0.1:8000
```

and:

```
http://127.0.0.1:8000/docs
```

For your coursework project

A minimal version with SQLite could look like this:

```
from fastapi import FastAPI
import sqlite3

app = FastAPI()

@app.get("/")
def root():
    return {"message": "Book API running"}

@app.get("/books")
def get_books():
    conn = sqlite3.connect("books.db")
    cursor = conn.cursor()
    cursor.execute("CREATE TABLE IF NOT EXISTS books (id INTEGER PRIMARY KEY, title
TEXT, author TEXT)")
    cursor.execute("SELECT * FROM books")
    books = cursor.fetchall()
    conn.close()
    return {"books": books}
```

Run with:

```
uvicorn main:app --reload
```

Fast way to check your problem

Look in your file and confirm these three things:

1. the file is named main.py
2. it contains app = FastAPI()
3. you are running uvicorn main:app --reload

If any one of those does not match, you get this error.

Paste your main.py here and I'll pinpoint the exact line causing it.

---+

Fix main.py with the actual database headers:

bookId title series author rating description language isbn genres characters bookFormat edition pages publisher publishDate firstPublishDate awards numRatings ratingsByStars likedPercent setting coverImg bbeScore bbeVotes price

--+
Since your CSV has **many columns**, your books table must match those headers exactly. Below is a **fixed main.py** that:

- creates a table with the correct schema
- imports your CSV safely
- exposes a FastAPI app
- avoids duplicate CSV imports
- supports CRUD endpoints

You can run it with:

```
uvicorn main:app --reload
```

Fixed

main.py

```
from fastapi import FastAPI, HTTPException
import sqlite3
import csv
import os

app = FastAPI()

DB_NAME = "books.db"


def get_connection():
    conn = sqlite3.connect(DB_NAME)
    conn.row_factory = sqlite3.Row
    return conn


def init_db():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    CREATE TABLE IF NOT EXISTS books (
        bookId INTEGER PRIMARY KEY,
        title TEXT,
        series TEXT,
        author TEXT,
        rating REAL,
```

```
        description TEXT,
        language TEXT,
        isbn TEXT,
        genres TEXT,
        characters TEXT,
        bookFormat TEXT,
        edition TEXT,
        pages INTEGER,
        publisher TEXT,
        publishDate TEXT,
        firstPublishDate TEXT,
        awards TEXT,
        numRatings INTEGER,
        ratingsByStars TEXT,
        likedPercent REAL,
        setting TEXT,
        coverImg TEXT,
        bbeScore REAL,
        bbeVotes INTEGER,
        price REAL
    )
    """)

    conn.commit()
    conn.close()


def import_csv():
    if not os.path.exists("books.csv"):
        return

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT COUNT(*) FROM books")
    count = cursor.fetchone()[0]

    if count == 0:
        with open("books.csv", newline="", encoding="utf-8") as file:
            reader = csv.DictReader(file)

            for row in reader:
                cursor.execute("""
                INSERT INTO books VALUES (
                    ?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?
                )
                """, (
                    row.get("bookId"),
                    row.get("title"),
                    row.get("series"),
                    row.get("author"),
                    row.get("rating"),
                    row.get("description"),
                    row.get("language"),
                    row.get("isbn"),
                    row.get("genres"),
                    row.get("characters"),
                    row.get("bookFormat"),
                    row.get("edition"),
                    row.get("pages"),
                    row.get("publisher"),
                    row.get("publishDate"),
                    row.get("firstPublishDate"),
                    row.get("awards"),
                    row.get("numRatings"),
                    row.get("ratingsByStars"),
                    row.get("likedPercent"),
```

```
                    row.get("setting"),
                    row.get("coverImg"),
                    row.get("bbeScore"),
                    row.get("bbeVotes"),
                    row.get("price")
                ))

    conn.commit()
    conn.close()


init_db()
import_csv()


@app.get("/")
def root():
    return {"message": "Book API running"}


@app.get("/books")
def get_books():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books LIMIT 100")
    books = [dict(row) for row in cursor.fetchall()]

    conn.close()

    return {"books": books}


@app.get("/books/{book_id}")
def get_book(book_id: int):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books WHERE bookId = ?", (book_id,))
    book = cursor.fetchone()

    conn.close()

    if book is None:
        raise HTTPException(status_code=404, detail="Book not found")

    return dict(book)


@app.post("/books", status_code=201)
def add_book(book: dict):
    conn = get_connection()
    cursor = conn.cursor()

    columns = ", ".join(book.keys())
    placeholders = ", ".join(["?"] * len(book))

    cursor.execute(
        f"INSERT INTO books ({columns}) VALUES ({placeholders})",
        tuple(book.values())
    )

    conn.commit()
    conn.close()

    return {"message": "Book added"}
```

```
@app.put("/books/{book_id}")
def update_book(book_id: int, book: dict):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books WHERE bookId = ?", (book_id,))
    existing = cursor.fetchone()

    if existing is None:
        conn.close()
        raise HTTPException(status_code=404, detail="Book not found")

    updates = ", ".join([f"{key}=?" for key in book.keys()])

    cursor.execute(
        f"UPDATE books SET {updates} WHERE bookId = ?",
        tuple(book.values()) + (book_id,)
    )

    conn.commit()
    conn.close()

    return {"message": "Book updated"}


@app.delete("/books/{book_id}", status_code=204)
def delete_book(book_id: int):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books WHERE bookId = ?", (book_id,))
    existing = cursor.fetchone()

    if existing is None:
        conn.close()
        raise HTTPException(status_code=404, detail="Book not found")

    cursor.execute("DELETE FROM books WHERE bookId = ?", (book_id,))

    conn.commit()
    conn.close()
```

Important things this fixes

1️⃣ Correct database schema

Matches your CSV headers exactly.

2️⃣ Prevents duplicate CSV imports

Only imports when table is empty.

3️⃣ Proper FastAPI app

Includes:

```
app = FastAPI()
```

so Uvicorn works.

4️⃣ Returns JSON properly

Uses:

```
dict(row)
```

so results serialize cleanly.

Test it

Run:

```
uvicorn main:app --reload
```

Open:

```
http://127.0.0.1:8000/docs
```

You should see endpoints:

- GET /books
- GET /books/{book_id}
- POST /books
- PUT /books/{book_id}
- DELETE /books/{book_id}

That satisfies the **CRUD + database + API endpoint requirements** for your assignment.

✅ If you want, I can also show you **3 really good analytics endpoints (genre trends, rating distribution, recommendations)** that will make your coursework look **much stronger than basic CRUD**.

---+

uvicorn main:app --reload

INFO: Will watch for changes in these directories: ['/Users/josephs/Desktop/Uni/WDWS/web_coursework_one']

INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)

INFO: Started reloader process [3696] using StatReload

Process SpawnProcess-1:

Traceback (most recent call last):

  File "/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/multiprocessing/process.py", line 314, in _bootstrap

    self.run()

  File "/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/multiprocessing/process.py", line 108, in run

    self._target(*self._args, **self._kwargs)

  File "/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/uvicorn/_subprocess.py", line 80, in subprocess_started

    target(sockets=sockets)

  File "/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/uvicorn/server.py", line 75, in run

    return asyncio_run(self.serve(sockets=sockets), loop_factory=self.config.get_loop_factory())

           ^^^

```
    return future.result()
           ^^^^^^^^^^^^^^^
  File "/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/uvicorn/server.py", line 79, in serve
    await self._serve(sockets)
  File "/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/uvicorn/server.py", line 86, in _serve
    config.load()
  File "/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/uvicorn/config.py", line 441, in load
    self.loaded_app = import_from_string(self.app)
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/uvicorn/importer.py", line 19, in import_from_string
    module = importlib.import_module(module_str)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/importlib/__init__.py", line 126, in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "<frozen importlib._bootstrap>", line 1204, in _gcd_import
  File "<frozen importlib._bootstrap>", line 1176, in _find_and_load
  File "<frozen importlib._bootstrap>", line 1147, in _find_and_load_unlocked
  File "<frozen importlib._bootstrap>", line 690, in _load_unlocked
  File "<frozen importlib._bootstrap_external>", line 940, in exec_module
  File "<frozen importlib._bootstrap>", line 241, in _call_with_frames_removed
  File "/Users/josephs/Desktop/Uni/WDWS/web_coursework_one/main.py", line 107, in <module>
    import_csv()
  File "/Users/josephs/Desktop/Uni/WDWS/web_coursework_one/main.py", line 70, in import_csv
    cursor.execute("""
```

sqlite3.OperationalError: table books has 3 columns but 25 values were supplied

---+
Your code is fine now. The problem is your **existing books.db file still contains the old books table** from earlier, when it only had 3 columns.

So when this runs:

```
INSERT INTO books VALUES (?, ?, ?, ..., ?)
```

SQLite is inserting **25 values** into a table that still only has **3 columns**.

Why this happened

CREATE TABLE IF NOT EXISTS books (...) does **not** replace an old table.

It means:

- if table does not exist, create it
- if table already exists, leave it alone

So SQLite kept your old 3-column version.

Fastest fix

Delete the old database file, then run Uvicorn again.

In your project folder:

```
rm books.db
uvicorn main:app --reload
```

On Mac, that should work.

If you prefer Finder, just manually delete books.db.

Why this fixes it

When the app starts again:

- init_db() creates a brand new books table
- this time with all 25 columns
- import_csv() can then insert the full CSV rows correctly

Confirm the old problem

Your old table was probably something like:

```
books(id, title, author)
```

But now your code expects:

```
books(
  bookId, title, series, author, rating, description, language, isbn,
  genres, characters, bookFormat, edition, pages, publisher, publishDate,
  firstPublishDate, awards, numRatings, ratingsByStars, likedPercent,
  setting, coverImg, bbeScore, bbeVotes, price
)
```

That mismatch causes the error.

Better fix inside

main.py

To avoid this in future, you can temporarily force a rebuild of the table while developing.

Replace your init_db() with this:

```
def init_db():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("DROP TABLE IF EXISTS books")

    cursor.execute("""
    CREATE TABLE books (
        bookId INTEGER PRIMARY KEY,
```

```
        title TEXT,
        series TEXT,
        author TEXT,
        rating REAL,
        description TEXT,
        language TEXT,
        isbn TEXT,
        genres TEXT,
        characters TEXT,
        bookFormat TEXT,
        edition TEXT,
        pages INTEGER,
        publisher TEXT,
        publishDate TEXT,
        firstPublishDate TEXT,
        awards TEXT,
        numRatings INTEGER,
        ratingsByStars TEXT,
        likedPercent REAL,
        setting TEXT,
        coverImg TEXT,
        bbeScore REAL,
        bbeVotes INTEGER,
        price REAL
    )
    """)

    conn.commit()
    conn.close()
```

This will recreate the table every time the app starts.

That is useful while testing, but later you may want to remove the DROP TABLE line so you do not erase data on every restart.

Best approach right now

Do this:

```
rm books.db
uvicorn main:app --reload
```

If you still get an error after that, paste your current main.py and I'll fix the exact line.

---+

uvicorn main:app --reload

INFO: Will watch for changes in these directories: ['/Users/josephs/Desktop/Uni/WDWS/web_coursework_one']

INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)

INFO: Started reloader process [3742] using StatReload

```
Process SpawnProcess-1:

Traceback (most recent call last):

  File "/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/multiprocessing/process.py", line 314, in _bootstrap

    self.run()

  File "/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/multiprocessing/process.py", line 108, in run

    self._target(*self._args, **self._kwargs)

  File "/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/uvicorn/_subprocess.py", line 80, in subprocess_started

    target(sockets=sockets)

  File "/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/uvicorn/server.py", line 75, in run

    return asyncio_run(self.serve(sockets=sockets), loop_factory=self.config.get_loop_factory())

           ^^^
```

```
    config.load()

  File "/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/uvicorn/config.py", line 441, in load

    self.loaded_app = import_from_string(self.app)

                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

  File "/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/uvicorn/importer.py", line 19, in import_from_string

    module = importlib.import_module(module_str)

             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

  File "/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/importlib/__init__.py", line 126, in import_module

    return _bootstrap._gcd_import(name[level:], package, level)

           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

  File "<frozen importlib._bootstrap>", line 1204, in _gcd_import

  File "<frozen importlib._bootstrap>", line 1176, in _find_and_load

  File "<frozen importlib._bootstrap>", line 1147, in _find_and_load_unlocked

  File "<frozen importlib._bootstrap>", line 690, in _load_unlocked

  File "<frozen importlib._bootstrap_external>", line 940, in exec_module

  File "<frozen importlib._bootstrap>", line 241, in _call_with_frames_removed

  File "/Users/josephs/Desktop/Uni/WDWS/web_coursework_one/main.py", line 107, in <module>

    import_csv()

  File "/Users/josephs/Desktop/Uni/WDWS/web_coursework_one/main.py", line 70, in import_csv

    cursor.execute("""

sqlite3.IntegrityError: datatype mismatch
```

---+
Found it — the main culprit is **bookId**.

Your CSV does **not** have numeric IDs. It has values like:

```
2767052-the-hunger-games
2.Harry_Potter_and_the_Order_of_the_Phoenix
```

So this schema is wrong:

```
bookId INTEGER PRIMARY KEY
```

SQLite is trying to store text like 2767052-the-hunger-games in an integer primary key, and that causes the datatype mismatch.

What to change

Make bookId a TEXT PRIMARY KEY, not INTEGER.

You should also treat a few messy dataset fields as text unless you really want to sanitize every value. Public CSVs love being chaotic.

Use this

main.py

```
from fastapi import FastAPI, HTTPException
import sqlite3
import csv
import os

app = FastAPI()

DB_NAME = "books.db"


def get_connection():
    conn = sqlite3.connect(DB_NAME)
    conn.row_factory = sqlite3.Row
    return conn


def to_int(value):
    if value is None:
        return None
    value = str(value).strip().replace(",", "")
    if value == "":
        return None
    try:
        return int(float(value))
    except ValueError:
        return None


def to_float(value):
    if value is None:
        return None
    value = str(value).strip().replace(",", "").replace("£", "").replace("$", "")
    if value == "":
        return None
```

```
    try:
        return float(value)
    except ValueError:
        return None


def to_text(value):
    if value is None:
        return None
    value = str(value).strip()
    return value if value != "" else None


def init_db():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("DROP TABLE IF EXISTS books")

    cursor.execute("""
    CREATE TABLE books (
        bookId TEXT PRIMARY KEY,
        title TEXT,
        series TEXT,
        author TEXT,
        rating REAL,
        description TEXT,
        language TEXT,
        isbn TEXT,
        genres TEXT,
        characters TEXT,
        bookFormat TEXT,
        edition TEXT,
        pages INTEGER,
        publisher TEXT,
        publishDate TEXT,
        firstPublishDate TEXT,
        awards TEXT,
        numRatings INTEGER,
        ratingsByStars TEXT,
        likedPercent REAL,
        setting TEXT,
        coverImg TEXT,
        bbeScore REAL,
        bbeVotes INTEGER,
        price REAL
    )
    """)

    conn.commit()
    conn.close()


def import_csv():
    if not os.path.exists("books.csv"):
        return

    conn = get_connection()
    cursor = conn.cursor()

    with open("books.csv", newline="", encoding="utf-8") as file:
        reader = csv.DictReader(file)

        for row in reader:
            cursor.execute("""
            INSERT INTO books (
                bookId, title, series, author, rating, description, language,
```

```
                isbn, genres, characters, bookFormat, edition, pages, publisher,
                publishDate, firstPublishDate, awards, numRatings, ratingsByStars,
                likedPercent, setting, coverImg, bbeScore, bbeVotes, price
            )
            VALUES
(?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
            """, (
                to_text(row.get("bookId")),
                to_text(row.get("title")),
                to_text(row.get("series")),
                to_text(row.get("author")),
                to_float(row.get("rating")),
                to_text(row.get("description")),
                to_text(row.get("language")),
                to_text(row.get("isbn")),
                to_text(row.get("genres")),
                to_text(row.get("characters")),
                to_text(row.get("bookFormat")),
                to_text(row.get("edition")),
                to_int(row.get("pages")),
                to_text(row.get("publisher")),
                to_text(row.get("publishDate")),
                to_text(row.get("firstPublishDate")),
                to_text(row.get("awards")),
                to_int(row.get("numRatings")),
                to_text(row.get("ratingsByStars")),
                to_float(row.get("likedPercent")),
                to_text(row.get("setting")),
                to_text(row.get("coverImg")),
                to_float(row.get("bbeScore")),
                to_int(row.get("bbeVotes")),
                to_float(row.get("price"))
            ))

    conn.commit()
    conn.close()


init_db()
import_csv()


@app.get("/")
def root():
    return {"message": "Book API running"}


@app.get("/books")
def get_books():
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM books LIMIT 100")
    books = [dict(row) for row in cursor.fetchall()]
    conn.close()
    return {"books": books}


@app.get("/books/{book_id}")
def get_book(book_id: str):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM books WHERE bookId = ?", (book_id,))
    book = cursor.fetchone()
    conn.close()

    if book is None:
        raise HTTPException(status_code=404, detail="Book not found")
```

```
    return dict(book)


@app.post("/books", status_code=201)
def add_book(book: dict):
    conn = get_connection()
    cursor = conn.cursor()

    columns = ", ".join(book.keys())
    placeholders = ", ".join(["?"] * len(book))

    cursor.execute(
        f"INSERT INTO books ({columns}) VALUES ({placeholders})",
        tuple(book.values())
    )

    conn.commit()
    conn.close()

    return {"message": "Book added"}


@app.put("/books/{book_id}")
def update_book(book_id: str, book: dict):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books WHERE bookId = ?", (book_id,))
    existing = cursor.fetchone()

    if existing is None:
        conn.close()
        raise HTTPException(status_code=404, detail="Book not found")

    updates = ", ".join([f"{key}=?" for key in book.keys()])

    cursor.execute(
        f"UPDATE books SET {updates} WHERE bookId = ?",
        tuple(book.values()) + (book_id,)
    )

    conn.commit()
    conn.close()

    return {"message": "Book updated"}


@app.delete("/books/{book_id}", status_code=204)
def delete_book(book_id: str):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books WHERE bookId = ?", (book_id,))
    existing = cursor.fetchone()

    if existing is None:
        conn.close()
        raise HTTPException(status_code=404, detail="Book not found")

    cursor.execute("DELETE FROM books WHERE bookId = ?", (book_id,))
    conn.commit()
    conn.close()
```

Then do this

Delete the old database again:

```
rm books.db
```

Then run:

```
uvicorn main:app --reload
```

Why this should work now

Your CSV sample shows:

- bookId = text slug, so TEXT PRIMARY KEY
- pages, numRatings, bbeVotes = numeric-looking, converted safely
- rating, likedPercent, bbeScore, price = converted to floats
- list-like fields such as genres, characters, awards stay as text, which is totally fine

Tiny extra note

Because you now use string IDs, your endpoint URL will look like this:

```
/books/2767052-the-hunger-games
```

not /books/1.

That’s actually fine for coursework.

If you want, next I’ll turn this into a slightly cleaner version with **Pydantic models** so your Swagger docs look much nicer.

---+

Here's the full assignment brief, what do I need to improve to meet it

---+

Yep — with the full brief in view, you’re already on track for the **minimum pass**, but there are a few things you should tighten up if you want to be safely above that and not get clipped on deliverables. The key requirements are: one database-backed CRUD model, at least four HTTP endpoints, JSON responses, proper status codes, runnable code, plus the non-code deliverables: GitHub repo, README, API docs PDF, technical report with GenAI declaration, and presentation slides.

What you likely already meet

From the API work so far, you appear to have:

- a **database-backed books model**
- **CRUD endpoints**
- **JSON responses**
- local execution via **FastAPI/Uvicorn**
- at least four endpoints

That covers the bare technical pass threshold in section 3(a).

What you should improve next

1. Stop rebuilding the database on every startup

Right now your main.py has been using:

```
cursor.execute("DROP TABLE IF EXISTS books")
```

That is fine while debugging, but for submission it makes the app look fragile because every restart wipes data. For a coursework demo, that can raise questions about persistence.

Better approach:

- create the table with CREATE TABLE IF NOT EXISTS
- only import CSV when the table is empty
- do **not** drop the table automatically

That makes the database behave like a real persistent backend, which is much more in line with the brief's expectation of CRUD linked to a database.

---

2. Add proper request models instead of raw

dict

At the moment, your POST and PUT endpoints likely take book: dict. That works, but it looks rough in Swagger and weakens validation.

You should use **Pydantic models** like:

```
from pydantic import BaseModel
from typing import Optional

class BookCreate(BaseModel):
    bookId: Optional[str] = None
    title: str
    series: Optional[str] = None
    author: Optional[str] = None
    rating: Optional[float] = None
    description: Optional[str] = None
    language: Optional[str] = None
    isbn: Optional[str] = None
    genres: Optional[str] = None
    characters: Optional[str] = None
    bookFormat: Optional[str] = None
    edition: Optional[str] = None
    pages: Optional[int] = None
    publisher: Optional[str] = None
    publishDate: Optional[str] = None
    firstPublishDate: Optional[str] = None
    awards: Optional[str] = None
    numRatings: Optional[int] = None
    ratingsByStars: Optional[str] = None
    likedPercent: Optional[float] = None
    setting: Optional[str] = None
    coverImg: Optional[str] = None
    bbeScore: Optional[float] = None
    bbeVotes: Optional[int] = None
    price: Optional[float] = None
```

Then:

```
@app.post("/books", status_code=201)
def add_book(book: BookCreate):
```

Why this matters:

- better validation
- clearer Swagger docs
- easier to explain in the oral exam
- stronger “software engineering principles” signal

That pushes you beyond a shaky pass toward the 50–69 region. The brief explicitly rewards stronger design justification, documentation, and technical understanding.

---

3. Improve error handling and status codes

The brief explicitly asks for correct status/error codes.

You already use 404 for missing books and 201 for POST, which is good. I'd also make sure you handle these cases neatly:

- 400 Bad Request when PUT gets no valid fields
- 404 Not Found when a requested record doesn't exist
- 201 Created for POST
- 204 No Content for DELETE
- FastAPI's 422 Unprocessable Entity will automatically handle bad request bodies if you use Pydantic

That gives you a cleaner story in the report and viva.

---

4. Add at least 2–3 analytics endpoints

This is where your project goes from "basic CRUD" to "actually interesting book API," which is exactly the spirit of the brief's example project ideas. The brief explicitly mentions a book metadata API with analytics like **genre trends**, **rating distributions**, and **personalised suggestions**.

I'd add these:

GET /analytics/genre-trends

Returns counts of books grouped by genre.

GET /analytics/rating-distribution

Returns rating buckets, such as:

- under 3
- 3 to 4
- 4 to 4.5
- above 4.5

GET /recommendations/top-rated

Returns the top-rated books, maybe filtered by language or genre.

Even simple versions of those make your project look much more aligned with the brief's higher-level expectations around creativity and data use.

---

5. Split

main.py

into modules

For a pass, one file is okay. For a better mark, modularise.

A cleaner structure:

```
project/
├── main.py
├── database.py
├── models.py
├── schemas.py
├── crud.py
├── import_csv.py
├── books.db
├── books.csv
├── README.md
└── docs/
```

That helps with the rubric's "Code Quality & Architecture" criteria. The marking grid explicitly scores code quality and architecture separately from raw functionality.

---

6. Produce proper API documentation as a PDF

This one is not optional. Missing API docs is a fail component. The brief says you need documentation describing endpoints, parameters, responses, example requests/responses, and auth if applicable, and it must be referenced from the README as a PDF.

So do this:

- use FastAPI Swagger for live docs
- also export a **PDF API documentation**
- include:
  - endpoint list
  - methods
  - parameters
  - example JSON request bodies

    - example JSON responses
    - error codes

You can make this manually in Markdown/Word and export to PDF if needed.

---

7. Write a real README

The brief says missing README or non-runnable code makes that component a fail.

Your README should include:

- project overview
- dataset source and citation
- stack used and why
- setup instructions
- how to run locally
- how to access Swagger docs
- where the API docs PDF is
- where the technical report is
- example requests

Make it frictionless for the examiner.

---

8. Write the technical report carefully

This is another fail component if missing, and it must include a **GenAI declaration**.

Your report should cover:

- project aim
- dataset source and licence
- why FastAPI + SQLite
- API design choices
- schema decisions
- challenges you hit
- testing approach
- limitations
- future improvements
- GenAI tools used, for what, and with examples/logs appendix

This matters a lot because the brief is very explicit that undeclared AI use is misconduct, and declared, thoughtful use is rewarded.

---

9. Add evidence of testing

The rubric mentions testing and error handling, especially from 60+ upward.

At minimum, test:

- valid GET /books
- valid GET /books/{id}
- invalid GET with nonexistent ID returns 404
- valid POST
- valid PUT
- valid DELETE
- invalid POST body returns validation error

Even a short testing section in the report with screenshots from Swagger/Postman is enough to strengthen the submission.

---

10. Get your GitHub history looking real

The brief says examiners will inspect commit history, and limited history is associated with lower bands.

So avoid one giant “final upload” commit. Make sensible commits like:

- initial FastAPI setup
- added SQLite schema and CSV import
- implemented CRUD endpoints
- improved validation with Pydantic
- added analytics endpoints
- wrote README and docs
- added report and slides

That helps a lot in the oral exam because your development process looks genuine.

---

11. Consider basic authentication if you want 50+

The rubric for 50–59 mentions “basic authentication present,” and 60–69 mentions a well-documented API with authentication.

It doesn’t look like authentication is mandatory for a pass, but adding a simple API key header for write endpoints could push you upward.

Example:

- GET endpoints public
- POST/PUT/DELETE require X-API-Key

That’s enough to talk about security without overcomplicating things.

---

12. Prepare for the oral

This brief is not just “submit code and disappear.” A lot rides on the 10-minute oral exam and your ability to explain your design decisions.

You should be ready to explain:

- why REST, not GraphQL
- why FastAPI
- why SQLite
- why CSV import into DB rather than querying CSV directly
- why id is the API primary key while bookId is just imported dataset metadata
- how your error handling works
- what your limitations are

That will matter in Q&A.

Biggest gaps to fix first

If I were prioritising, I'd do these in order:

1. **Stabilise main.py** so it does not drop the DB every startup
2. **Add Pydantic request models**
3. **Add 2–3 analytics endpoints**
4. **Write README + API docs PDF**
5. **Write technical report with GenAI declaration**
6. **Make decent slides for the oral**

Where your project probably sits right now

Based on what you've shown, you are around:

- **Pass / low 50s** if the API runs, CRUD works, and you submit the required documents
- **60+** if you improve structure, docs, validation, testing, and error handling
- **70+** if you add cleaner modular architecture, analytics, good presentation, strong Git history, and confident viva explanations

My honest recommendation

Your quickest route to a solid mark is:

- keep the current book API
- clean the architecture a bit
- add three analytics endpoints
- add validation and docs
- produce polished supporting materials

That would line up really well with the brief's book API example and the higher rubric bands.

Paste your current main.py again and I'll mark it up line-by-line with exactly what to change for a safer 60+.

---+

```
from fastapi import FastAPI, HTTPException

import sqlite3

import csv
```

```
import os


app = FastAPI()


DB_NAME = "books.db"



def get_connection():
    conn = sqlite3.connect(DB_NAME)
    conn.row_factory = sqlite3.Row
    return conn



def to_int(value):
    if value is None:
        return None
    value = str(value).strip().replace(",", "")
    if value == "":
        return None
    try:
        return int(float(value))
    except ValueError:
        return None



def to_float(value):
    if value is None:
        return None
```

```
    value = str(value).strip().replace(",", "").replace("£", "").replace("$", "")
    if value == "":
        return None
    try:
        return float(value)
    except ValueError:
        return None



def to_text(value):
    if value is None:
        return None
    value = str(value).strip()
    return value if value != "" else None



def init_db():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("DROP TABLE IF EXISTS books")

    cursor.execute("""
    CREATE TABLE books (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        bookId TEXT,
        title TEXT,
        series TEXT,
```

```
    author TEXT,
    rating REAL,
    description TEXT,
    language TEXT,
    isbn TEXT,
    genres TEXT,
    characters TEXT,
    bookFormat TEXT,
    edition TEXT,
    pages INTEGER,
    publisher TEXT,
    publishDate TEXT,
    firstPublishDate TEXT,
    awards TEXT,
    numRatings INTEGER,
    ratingsByStars TEXT,
    likedPercent REAL,
    setting TEXT,
    coverImg TEXT,
    bbeScore REAL,
    bbeVotes INTEGER,
    price REAL
)
""")

conn.commit()
conn.close()
```

```
def import_csv():
    if not os.path.exists("books.csv"):
        return

    conn = get_connection()
    cursor = conn.cursor()

    with open("books.csv", newline="", encoding="utf-8") as file:
        reader = csv.DictReader(file)

        for row in reader:
            cursor.execute("""
            INSERT INTO books (
                bookId, title, series, author, rating, description, language,
                isbn, genres, characters, bookFormat, edition, pages, publisher,
                publishDate, firstPublishDate, awards, numRatings, ratingsByStars,
                likedPercent, setting, coverImg, bbeScore, bbeVotes, price
            )
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
            """, (
                to_text(row.get("bookId")),
                to_text(row.get("title")),
                to_text(row.get("series")),
                to_text(row.get("author")),
                to_float(row.get("rating")),
                to_text(row.get("description")),
                to_text(row.get("language")),
```

```
            to_text(row.get("isbn")),
            to_text(row.get("genres")),
            to_text(row.get("characters")),
            to_text(row.get("bookFormat")),
            to_text(row.get("edition")),
            to_int(row.get("pages")),
            to_text(row.get("publisher")),
            to_text(row.get("publishDate")),
            to_text(row.get("firstPublishDate")),
            to_text(row.get("awards")),
            to_int(row.get("numRatings")),
            to_text(row.get("ratingsByStars")),
            to_float(row.get("likedPercent")),
            to_text(row.get("setting")),
            to_text(row.get("coverImg")),
            to_float(row.get("bbeScore")),
            to_int(row.get("bbeVotes")),
            to_float(row.get("price"))
        ))

    conn.commit()
    conn.close()


init_db()
import_csv()
```

```
@app.get("/")
def root():
    return {"message": "Book API running"}



@app.get("/books")
def get_books():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books LIMIT 100")
    books = [dict(row) for row in cursor.fetchall()]

    conn.close()
    return {"books": books}



@app.get("/books/{id}")
def get_book(id: int):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books WHERE id = ?", (id,))
    book = cursor.fetchone()

    conn.close()

    if book is None:
```

```
        raise HTTPException(status_code=404, detail="Book not found")

    return dict(book)



@app.post("/books", status_code=201)
def add_book(book: dict):
    conn = get_connection()
    cursor = conn.cursor()

    allowed_fields = [
        "bookId","title","series","author","rating","description","language",
        "isbn","genres","characters","bookFormat","edition","pages","publisher",
        "publishDate","firstPublishDate","awards","numRatings","ratingsByStars",
        "likedPercent","setting","coverImg","bbeScore","bbeVotes","price"
    ]

    filtered_book = {k: v for k, v in book.items() if k in allowed_fields}

    columns = ", ".join(filtered_book.keys())
    placeholders = ", ".join(["?"] * len(filtered_book))

    cursor.execute(
        f"INSERT INTO books ({columns}) VALUES ({placeholders})",
        tuple(filtered_book.values())
    )

    conn.commit()
```

```
    new_id = cursor.lastrowid
    conn.close()

    return {"message": "Book added", "id": new_id}



@app.put("/books/{id}")
def update_book(id: int, book: dict):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books WHERE id = ?", (id,))
    existing = cursor.fetchone()

    if existing is None:
        conn.close()
        raise HTTPException(status_code=404, detail="Book not found")

    updates = ", ".join([f"{key}=?" for key in book.keys()])

    cursor.execute(
        f"UPDATE books SET {updates} WHERE id = ?",
        tuple(book.values()) + (id,)
    )

    conn.commit()
    conn.close()
```

```
    return {"message": "Book updated"}



@app.delete("/books/{id}", status_code=204)
def delete_book(id: int):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books WHERE id = ?", (id,))
    existing = cursor.fetchone()

    if existing is None:
        conn.close()
        raise HTTPException(status_code=404, detail="Book not found")

    cursor.execute("DELETE FROM books WHERE id = ?", (id,))
    conn.commit()
    conn.close()
```

---+

Yep — this is a decent working base, but there are **four weak spots** in this exact file that I'd fix first if you want a safer **60+**.

The biggest issues in your current main.py

1. You wipe the database every time

This is the biggest one.

You currently have:

```
cursor.execute("DROP TABLE IF EXISTS books")
```

That means every restart deletes all data, which makes CRUD look fake rather than persistent.

Replace your init_db() with this:

```
def init_db():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    CREATE TABLE IF NOT EXISTS books (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        bookId TEXT,
        title TEXT NOT NULL,
        series TEXT,
        author TEXT,
        rating REAL,
        description TEXT,
        language TEXT,
        isbn TEXT,
        genres TEXT,
        characters TEXT,
        bookFormat TEXT,
        edition TEXT,
        pages INTEGER,
        publisher TEXT,
        publishDate TEXT,
        firstPublishDate TEXT,
        awards TEXT,
        numRatings INTEGER,
        ratingsByStars TEXT,
        likedPercent REAL,
        setting TEXT,
        coverImg TEXT,
        bbeScore REAL,
        bbeVotes INTEGER,
        price REAL
    )
    """)

    conn.commit()
    conn.close()
```

Then change import_csv() so it only imports when the table is empty:

```
def import_csv():
    if not os.path.exists("books.csv"):
        return

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT COUNT(*) FROM books")
    count = cursor.fetchone()[0]

    if count == 0:
        with open("books.csv", newline="", encoding="utf-8") as file:
```

```
            reader = csv.DictReader(file)

            for row in reader:
                cursor.execute("""
                INSERT INTO books (
                    bookId, title, series, author, rating, description, language,
                    isbn, genres, characters, bookFormat, edition, pages, publisher,
                    publishDate, firstPublishDate, awards, numRatings, ratingsByStars,
                    likedPercent, setting, coverImg, bbeScore, bbeVotes, price
                )
                VALUES
(?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
                """, (
                    to_text(row.get("bookId")),
                    to_text(row.get("title")),
                    to_text(row.get("series")),
                    to_text(row.get("author")),
                    to_float(row.get("rating")),
                    to_text(row.get("description")),
                    to_text(row.get("language")),
                    to_text(row.get("isbn")),
                    to_text(row.get("genres")),
                    to_text(row.get("characters")),
                    to_text(row.get("bookFormat")),
                    to_text(row.get("edition")),
                    to_int(row.get("pages")),
                    to_text(row.get("publisher")),
                    to_text(row.get("publishDate")),
                    to_text(row.get("firstPublishDate")),
                    to_text(row.get("awards")),
                    to_int(row.get("numRatings")),
                    to_text(row.get("ratingsByStars")),
                    to_float(row.get("likedPercent")),
                    to_text(row.get("setting")),
                    to_text(row.get("coverImg")),
                    to_float(row.get("bbeScore")),
                    to_int(row.get("bbeVotes")),
                    to_float(row.get("price"))
                ))

    conn.commit()
    conn.close()
```

That aligns much better with the requirement for database-backed CRUD rather than re-seeding from scratch on every run.

---

2. Your POST and PUT use raw

dict

This works, but it’s pretty rough. Swagger looks worse, validation is weaker, and markers will spot that.

Add this near the top:

```
from pydantic import BaseModel
from typing import Optional
```

Then add these schemas:

```
class BookCreate(BaseModel):
    bookId: Optional[str] = None
    title: str
    series: Optional[str] = None
    author: Optional[str] = None
    rating: Optional[float] = None
    description: Optional[str] = None
    language: Optional[str] = None
    isbn: Optional[str] = None
    genres: Optional[str] = None
    characters: Optional[str] = None
    bookFormat: Optional[str] = None
    edition: Optional[str] = None
    pages: Optional[int] = None
    publisher: Optional[str] = None
    publishDate: Optional[str] = None
    firstPublishDate: Optional[str] = None
    awards: Optional[str] = None
    numRatings: Optional[int] = None
    ratingsByStars: Optional[str] = None
    likedPercent: Optional[float] = None
    setting: Optional[str] = None
    coverImg: Optional[str] = None
    bbeScore: Optional[float] = None
    bbeVotes: Optional[int] = None
    price: Optional[float] = None


class BookUpdate(BaseModel):
    bookId: Optional[str] = None
    title: Optional[str] = None
    series: Optional[str] = None
    author: Optional[str] = None
    rating: Optional[float] = None
    description: Optional[str] = None
    language: Optional[str] = None
    isbn: Optional[str] = None
    genres: Optional[str] = None
    characters: Optional[str] = None
    bookFormat: Optional[str] = None
    edition: Optional[str] = None
    pages: Optional[int] = None
    publisher: Optional[str] = None
    publishDate: Optional[str] = None
    firstPublishDate: Optional[str] = None
    awards: Optional[str] = None
    numRatings: Optional[int] = None
    ratingsByStars: Optional[str] = None
    likedPercent: Optional[float] = None
    setting: Optional[str] = None
    coverImg: Optional[str] = None
    bbeScore: Optional[float] = None
    bbeVotes: Optional[int] = None
    price: Optional[float] = None
```

Then replace your POST endpoint with:

```
@app.post("/books", status_code=201)
def add_book(book: BookCreate):
    conn = get_connection()
    cursor = conn.cursor()

    data = book.model_dump(exclude_none=True)
```

```
    columns = ", ".join(data.keys())
    placeholders = ", ".join(["?"] * len(data))

    cursor.execute(
        f"INSERT INTO books ({columns}) VALUES ({placeholders})",
        tuple(data.values())
    )

    conn.commit()
    new_id = cursor.lastrowid
    conn.close()

    return {"message": "Book added", "id": new_id}
```

And replace your PUT endpoint with:

```
@app.put("/books/{id}")
def update_book(id: int, book: BookUpdate):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books WHERE id = ?", (id,))
    existing = cursor.fetchone()

    if existing is None:
        conn.close()
        raise HTTPException(status_code=404, detail="Book not found")

    data = book.model_dump(exclude_none=True)

    if not data:
        conn.close()
        raise HTTPException(status_code=400, detail="No fields provided for update")

    updates = ", ".join([f"{key}=?" for key in data.keys()])

    cursor.execute(
        f"UPDATE books SET {updates} WHERE id = ?",
        tuple(data.values()) + (id,)
    )

    conn.commit()
    conn.close()

    return {"message": "Book updated"}
```

This gives you proper validation and much nicer interactive docs, which helps with both the technical quality and the required documentation side.

---

3. Your

DELETE

says 204 but returns a body

This is a sneaky one.

You currently have:

```
@app.delete("/books/{id}", status_code=204)
def delete_book(id: int):
```

but you don't return anything. That's okay. The issue is: if you later return a message body with 204, that would be wrong. So keep it empty on success.

Make it explicit:

```
from fastapi import Response
```

Then:

```
@app.delete("/books/{id}", status_code=204)
def delete_book(id: int):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books WHERE id = ?", (id,))
    existing = cursor.fetchone()

    if existing is None:
        conn.close()
        raise HTTPException(status_code=404, detail="Book not found")

    cursor.execute("DELETE FROM books WHERE id = ?", (id,))
    conn.commit()
    conn.close()

    return Response(status_code=204)
```

That's cleaner and fully consistent with HTTP conventions the brief asks for.

---

4. You need at least a couple of analytics endpoints

Your CRUD is enough for the minimum pass, but the brief's example project idea for books explicitly mentions **genre trends, rating distributions, and personalised suggestions**. Adding even simple versions makes your API look much more aligned with the assignment.

Add these three:

```
@app.get("/analytics/genre-trends")
def genre_trends():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
        SELECT genres, COUNT(*) as count
        FROM books
        WHERE genres IS NOT NULL AND genres != ''
        GROUP BY genres
        ORDER BY count DESC
```

```
        LIMIT 10
    """)
    results = [dict(row) for row in cursor.fetchall()]
    conn.close()

    return {"genre_trends": results}
@app.get("/analytics/top-rated")
def top_rated():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
        SELECT id, title, author, rating
        FROM books
        WHERE rating IS NOT NULL
        ORDER BY rating DESC
        LIMIT 10
    """)
    results = [dict(row) for row in cursor.fetchall()]
    conn.close()

    return {"top_rated": results}
@app.get("/analytics/rating-distribution")
def rating_distribution():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
        SELECT
            CASE
                WHEN rating < 3 THEN 'Below 3'
                WHEN rating >= 3 AND rating < 4 THEN '3 to 3.9'
                WHEN rating >= 4 AND rating < 4.5 THEN '4 to 4.4'
                ELSE '4.5 and above'
            END AS rating_band,
            COUNT(*) AS count
        FROM books
        WHERE rating IS NOT NULL
        GROUP BY rating_band
        ORDER BY count DESC
    """)
    results = [dict(row) for row in cursor.fetchall()]
    conn.close()

    return {"rating_distribution": results}
```

They’re simple, but they make the project feel like a real book data API rather than just a database exercise.

One full improved version

This is the version I’d actually submit as your next step:

```
from fastapi import FastAPI, HTTPException, Response
from pydantic import BaseModel
from typing import Optional
import sqlite3
import csv
import os

app = FastAPI(title="Book Metadata API")
```

```
DB_NAME = "books.db"


class BookCreate(BaseModel):
    bookId: Optional[str] = None
    title: str
    series: Optional[str] = None
    author: Optional[str] = None
    rating: Optional[float] = None
    description: Optional[str] = None
    language: Optional[str] = None
    isbn: Optional[str] = None
    genres: Optional[str] = None
    characters: Optional[str] = None
    bookFormat: Optional[str] = None
    edition: Optional[str] = None
    pages: Optional[int] = None
    publisher: Optional[str] = None
    publishDate: Optional[str] = None
    firstPublishDate: Optional[str] = None
    awards: Optional[str] = None
    numRatings: Optional[int] = None
    ratingsByStars: Optional[str] = None
    likedPercent: Optional[float] = None
    setting: Optional[str] = None
    coverImg: Optional[str] = None
    bbeScore: Optional[float] = None
    bbeVotes: Optional[int] = None
    price: Optional[float] = None


class BookUpdate(BaseModel):
    bookId: Optional[str] = None
    title: Optional[str] = None
    series: Optional[str] = None
    author: Optional[str] = None
    rating: Optional[float] = None
    description: Optional[str] = None
    language: Optional[str] = None
    isbn: Optional[str] = None
    genres: Optional[str] = None
    characters: Optional[str] = None
    bookFormat: Optional[str] = None
    edition: Optional[str] = None
    pages: Optional[int] = None
    publisher: Optional[str] = None
    publishDate: Optional[str] = None
    firstPublishDate: Optional[str] = None
    awards: Optional[str] = None
    numRatings: Optional[int] = None
    ratingsByStars: Optional[str] = None
    likedPercent: Optional[float] = None
    setting: Optional[str] = None
    coverImg: Optional[str] = None
    bbeScore: Optional[float] = None
    bbeVotes: Optional[int] = None
    price: Optional[float] = None


def get_connection():
    conn = sqlite3.connect(DB_NAME)
    conn.row_factory = sqlite3.Row
    return conn


def to_int(value):
```

```
    if value is None:
        return None
    value = str(value).strip().replace(",", "")
    if value == "":
        return None
    try:
        return int(float(value))
    except ValueError:
        return None


def to_float(value):
    if value is None:
        return None
    value = str(value).strip().replace(",", "").replace("£", "").replace("$", "")
    if value == "":
        return None
    try:
        return float(value)
    except ValueError:
        return None


def to_text(value):
    if value is None:
        return None
    value = str(value).strip()
    return value if value != "" else None


def init_db():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    CREATE TABLE IF NOT EXISTS books (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        bookId TEXT,
        title TEXT NOT NULL,
        series TEXT,
        author TEXT,
        rating REAL,
        description TEXT,
        language TEXT,
        isbn TEXT,
        genres TEXT,
        characters TEXT,
        bookFormat TEXT,
        edition TEXT,
        pages INTEGER,
        publisher TEXT,
        publishDate TEXT,
        firstPublishDate TEXT,
        awards TEXT,
        numRatings INTEGER,
        ratingsByStars TEXT,
        likedPercent REAL,
        setting TEXT,
        coverImg TEXT,
        bbeScore REAL,
        bbeVotes INTEGER,
        price REAL
    )
    """)

    conn.commit()
    conn.close()
```

```
def import_csv():
    if not os.path.exists("books.csv"):
        return

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT COUNT(*) FROM books")
    count = cursor.fetchone()[0]

    if count == 0:
        with open("books.csv", newline="", encoding="utf-8") as file:
            reader = csv.DictReader(file)

            for row in reader:
                cursor.execute("""
                INSERT INTO books (
                    bookId, title, series, author, rating, description, language,
                    isbn, genres, characters, bookFormat, edition, pages, publisher,
                    publishDate, firstPublishDate, awards, numRatings, ratingsByStars,
                    likedPercent, setting, coverImg, bbeScore, bbeVotes, price
                )
                VALUES
(?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
                """, (
                    to_text(row.get("bookId")),
                    to_text(row.get("title")),
                    to_text(row.get("series")),
                    to_text(row.get("author")),
                    to_float(row.get("rating")),
                    to_text(row.get("description")),
                    to_text(row.get("language")),
                    to_text(row.get("isbn")),
                    to_text(row.get("genres")),
                    to_text(row.get("characters")),
                    to_text(row.get("bookFormat")),
                    to_text(row.get("edition")),
                    to_int(row.get("pages")),
                    to_text(row.get("publisher")),
                    to_text(row.get("publishDate")),
                    to_text(row.get("firstPublishDate")),
                    to_text(row.get("awards")),
                    to_int(row.get("numRatings")),
                    to_text(row.get("ratingsByStars")),
                    to_float(row.get("likedPercent")),
                    to_text(row.get("setting")),
                    to_text(row.get("coverImg")),
                    to_float(row.get("bbeScore")),
                    to_int(row.get("bbeVotes")),
                    to_float(row.get("price"))
                ))

    conn.commit()
    conn.close()


init_db()
import_csv()


@app.get("/")
def root():
    return {"message": "Book API running"}
```

```python
@app.get("/books")
def get_books():
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM books LIMIT 100")
    books = [dict(row) for row in cursor.fetchall()]
    conn.close()
    return {"books": books}


@app.get("/books/{id}")
def get_book(id: int):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM books WHERE id = ?", (id,))
    book = cursor.fetchone()
    conn.close()

    if book is None:
        raise HTTPException(status_code=404, detail="Book not found")

    return dict(book)


@app.post("/books", status_code=201)
def add_book(book: BookCreate):
    conn = get_connection()
    cursor = conn.cursor()

    data = book.model_dump(exclude_none=True)
    columns = ", ".join(data.keys())
    placeholders = ", ".join(["?"] * len(data))

    cursor.execute(
        f"INSERT INTO books ({columns}) VALUES ({placeholders})",
        tuple(data.values())
    )

    conn.commit()
    new_id = cursor.lastrowid
    conn.close()

    return {"message": "Book added", "id": new_id}


@app.put("/books/{id}")
def update_book(id: int, book: BookUpdate):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books WHERE id = ?", (id,))
    existing = cursor.fetchone()

    if existing is None:
        conn.close()
        raise HTTPException(status_code=404, detail="Book not found")

    data = book.model_dump(exclude_none=True)

    if not data:
        conn.close()
        raise HTTPException(status_code=400, detail="No fields provided for update")

    updates = ", ".join([f"{key}=?" for key in data.keys()])

    cursor.execute(
        f"UPDATE books SET {updates} WHERE id = ?",
```

```
        tuple(data.values()) + (id,)
    )

    conn.commit()
    conn.close()

    return {"message": "Book updated"}


@app.delete("/books/{id}", status_code=204)
def delete_book(id: int):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books WHERE id = ?", (id,))
    existing = cursor.fetchone()

    if existing is None:
        conn.close()
        raise HTTPException(status_code=404, detail="Book not found")

    cursor.execute("DELETE FROM books WHERE id = ?", (id,))
    conn.commit()
    conn.close()

    return Response(status_code=204)


@app.get("/analytics/genre-trends")
def genre_trends():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
        SELECT genres, COUNT(*) as count
        FROM books
        WHERE genres IS NOT NULL AND genres != ''
        GROUP BY genres
        ORDER BY count DESC
        LIMIT 10
    """)
    results = [dict(row) for row in cursor.fetchall()]
    conn.close()

    return {"genre_trends": results}


@app.get("/analytics/top-rated")
def top_rated():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
        SELECT id, title, author, rating
        FROM books
        WHERE rating IS NOT NULL
        ORDER BY rating DESC
        LIMIT 10
    """)
    results = [dict(row) for row in cursor.fetchall()]
    conn.close()

    return {"top_rated": results}


@app.get("/analytics/rating-distribution")
def rating_distribution():
```

```
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
        SELECT
            CASE
                WHEN rating < 3 THEN 'Below 3'
                WHEN rating >= 3 AND rating < 4 THEN '3 to 3.9'
                WHEN rating >= 4 AND rating < 4.5 THEN '4 to 4.4'
                ELSE '4.5 and above'
            END AS rating_band,
            COUNT(*) AS count
        FROM books
        WHERE rating IS NOT NULL
        GROUP BY rating_band
        ORDER BY count DESC
    """)
    results = [dict(row) for row in cursor.fetchall()]
    conn.close()

    return {"rating_distribution": results}
```

What this gets you

With just those changes, you'll be in much better shape for:

- proper persistent CRUD
- cleaner docs
- better validation
- stronger status code usage
- endpoints beyond the bare minimum

That matches the brief much more comfortably than the current version.

After this file, your next priorities are

1. write a strong README
2. create a PDF of API docs
3. write the technical report with GenAI declaration
4. add a few screenshots/tests
5. make simple presentation slides

Those are all explicitly part of the submission package, not just "nice extras."

Paste your README draft next and I'll tighten it up so it sounds properly coursework-grade.

---+

Without changing the code at all, can you add some clean commenting to my main.py?

```
from fastapi import FastAPI, HTTPException, Response
import sqlite3
import csv
import os
from pydantic import BaseModel
from typing import Optional

class BookCreate(BaseModel):
    bookId: Optional[str] = None
    title: str
    series: Optional[str] = None
    author: Optional[str] = None
    rating: Optional[float] = None
    description: Optional[str] = None
    language: Optional[str] = None
    isbn: Optional[str] = None
    genres: Optional[str] = None
    characters: Optional[str] = None
    bookFormat: Optional[str] = None
    edition: Optional[str] = None
    pages: Optional[int] = None
    publisher: Optional[str] = None
    publishDate: Optional[str] = None
    firstPublishDate: Optional[str] = None
    awards: Optional[str] = None
    numRatings: Optional[int] = None
```

```
    ratingsByStars: Optional[str] = None
    likedPercent: Optional[float] = None
    setting: Optional[str] = None
    coverImg: Optional[str] = None
    bbeScore: Optional[float] = None
    bbeVotes: Optional[int] = None
    price: Optional[float] = None


class BookUpdate(BaseModel):
    bookId: Optional[str] = None
    title: Optional[str] = None
    series: Optional[str] = None
    author: Optional[str] = None
    rating: Optional[float] = None
    description: Optional[str] = None
    language: Optional[str] = None
    isbn: Optional[str] = None
    genres: Optional[str] = None
    characters: Optional[str] = None
    bookFormat: Optional[str] = None
    edition: Optional[str] = None
    pages: Optional[int] = None
    publisher: Optional[str] = None
    publishDate: Optional[str] = None
    firstPublishDate: Optional[str] = None
    awards: Optional[str] = None
    numRatings: Optional[int] = None
```

```
    ratingsByStars: Optional[str] = None
    likedPercent: Optional[float] = None
    setting: Optional[str] = None
    coverImg: Optional[str] = None
    bbeScore: Optional[float] = None
    bbeVotes: Optional[int] = None
    price: Optional[float] = None

app = FastAPI()

DB_NAME = "books.db"


def get_connection():
    conn = sqlite3.connect(DB_NAME)
    conn.row_factory = sqlite3.Row
    return conn


def to_int(value):
    if value is None:
        return None
    value = str(value).strip().replace(",", "")
    if value == "":
        return None
    try:
        return int(float(value))
    except ValueError:
```

```
        return None



def to_float(value):
    if value is None:
        return None
    value = str(value).strip().replace(",", "").replace("£", "").replace("$", "")
    if value == "":
        return None
    try:
        return float(value)
    except ValueError:
        return None



def to_text(value):
    if value is None:
        return None
    value = str(value).strip()
    return value if value != "" else None



def init_db():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    CREATE TABLE IF NOT EXISTS books (
```

```
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    bookId TEXT,
    title TEXT NOT NULL,
    series TEXT,
    author TEXT,
    rating REAL,
    description TEXT,
    language TEXT,
    isbn TEXT,
    genres TEXT,
    characters TEXT,
    bookFormat TEXT,
    edition TEXT,
    pages INTEGER,
    publisher TEXT,
    publishDate TEXT,
    firstPublishDate TEXT,
    awards TEXT,
    numRatings INTEGER,
    ratingsByStars TEXT,
    likedPercent REAL,
    setting TEXT,
    coverImg TEXT,
    bbeScore REAL,
    bbeVotes INTEGER,
    price REAL
)
""")
```

```
    conn.commit()
    conn.close()



def import_csv():
    if not os.path.exists("books.csv"):
        return

    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT COUNT(*) FROM books")
    count = cursor.fetchone()[0]

    if count == 0:
        with open("books.csv", newline="", encoding="utf-8") as file:
            reader = csv.DictReader(file)

            for row in reader:
                cursor.execute("""
                INSERT INTO books (
                    bookId, title, series, author, rating, description, language,
                    isbn, genres, characters, bookFormat, edition, pages, publisher,
                    publishDate, firstPublishDate, awards, numRatings, ratingsByStars,
                    likedPercent, setting, coverImg, bbeScore, bbeVotes, price
                )
                VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

```
""", (
    to_text(row.get("bookId")),
    to_text(row.get("title")),
    to_text(row.get("series")),
    to_text(row.get("author")),
    to_float(row.get("rating")),
    to_text(row.get("description")),
    to_text(row.get("language")),
    to_text(row.get("isbn")),
    to_text(row.get("genres")),
    to_text(row.get("characters")),
    to_text(row.get("bookFormat")),
    to_text(row.get("edition")),
    to_int(row.get("pages")),
    to_text(row.get("publisher")),
    to_text(row.get("publishDate")),
    to_text(row.get("firstPublishDate")),
    to_text(row.get("awards")),
    to_int(row.get("numRatings")),
    to_text(row.get("ratingsByStars")),
    to_float(row.get("likedPercent")),
    to_text(row.get("setting")),
    to_text(row.get("coverImg")),
    to_float(row.get("bbeScore")),
    to_int(row.get("bbeVotes")),
    to_float(row.get("price"))
))
```

```
    conn.commit()
    conn.close()



init_db()
import_csv()



@app.get("/")
def root():
    return {"message": "Book API running"}



@app.get("/books")
def get_books():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books LIMIT 100")
    books = [dict(row) for row in cursor.fetchall()]

    conn.close()
    return {"books": books}



@app.get("/books/{id}")
def get_book(id: int):
    conn = get_connection()
```

```
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books WHERE id = ?", (id,))
    book = cursor.fetchone()

    conn.close()

    if book is None:
        raise HTTPException(status_code=404, detail="Book not found")

    return dict(book)



@app.post("/books", status_code=201)
def add_book(book: BookCreate):
    conn = get_connection()
    cursor = conn.cursor()

    data = book.model_dump(exclude_none=True)

    columns = ", ".join(data.keys())
    placeholders = ", ".join(["?"] * len(data))

    cursor.execute(
        f"INSERT INTO books ({columns}) VALUES ({placeholders})",
        tuple(data.values())
    )
```

```
    conn.commit()
    new_id = cursor.lastrowid
    conn.close()

    return {"message": "Book added", "id": new_id}



@app.put("/books/{id}")
def update_book(id: int, book: BookUpdate):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books WHERE id = ?", (id,))
    existing = cursor.fetchone()

    if existing is None:
        conn.close()
        raise HTTPException(status_code=404, detail="Book not found")

    data = book.model_dump(exclude_none=True)

    if not data:
        conn.close()
        raise HTTPException(status_code=400, detail="No fields provided for update")

    updates = ", ".join([f"{key}=?" for key in data.keys()])

    cursor.execute(
```

```
        f"UPDATE books SET {updates} WHERE id = ?",
        tuple(data.values()) + (id,)
    )

    conn.commit()
    conn.close()

    return {"message": "Book updated"}



@app.delete("/books/{id}", status_code=204)
def delete_book(id: int):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books WHERE id = ?", (id,))
    existing = cursor.fetchone()

    if existing is None:
        conn.close()
        raise HTTPException(status_code=404, detail="Book not found")

    cursor.execute("DELETE FROM books WHERE id = ?", (id,))
    conn.commit()
    conn.close()

    return Response(status_code=204)
```

```
@app.get("/analytics/genre-trends")
def genre_trends():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
        SELECT genres, COUNT(*) as count
        FROM books
        WHERE genres IS NOT NULL AND genres != ''
        GROUP BY genres
        ORDER BY count DESC
        LIMIT 10
    """)
    results = [dict(row) for row in cursor.fetchall()]
    conn.close()

    return {"genre_trends": results}

@app.get("/analytics/top-rated")
def top_rated():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
        SELECT id, title, author, rating
        FROM books
        WHERE rating IS NOT NULL
        ORDER BY rating DESC
```

```
        LIMIT 10
    """)
    results = [dict(row) for row in cursor.fetchall()]
    conn.close()

    return {"top_rated": results}

@app.get("/analytics/rating-distribution")
def rating_distribution():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
        SELECT
            CASE
                WHEN rating < 3 THEN 'Below 3'
                WHEN rating >= 3 AND rating < 4 THEN '3 to 3.9'
                WHEN rating >= 4 AND rating < 4.5 THEN '4 to 4.4'
                ELSE '4.5 and above'
            END AS rating_band,
            COUNT(*) AS count
        FROM books
        WHERE rating IS NOT NULL
        GROUP BY rating_band
        ORDER BY count DESC
    """)
    results = [dict(row) for row in cursor.fetchall()]
    conn.close()
```

return {"rating_distribution": results}

---+
Below is **your exact code with clean, coursework-appropriate comments added**.

I **did not change any logic or structure**, only added:

- file-level explanation
- section headers
- function descriptions
- short inline explanations where useful

This level of commenting is usually what markers expect: **clear but not excessive**.

```
# ------------------------------------------------------------
# Book Metadata and Recommendation API
# COMP3011 Web Services and Web Data Coursework
#
# This FastAPI application provides CRUD functionality for a
# book metadata dataset stored in an SQLite database.
#
# The database is initially populated from a CSV dataset.
# Additional analytics endpoints provide insights into the
# dataset, such as genre trends and rating distributions.
# ------------------------------------------------------------

from fastapi import FastAPI, HTTPException, Response
import sqlite3
import csv
import os
from pydantic import BaseModel
from typing import Optional


# ------------------------------------------------------------
# Pydantic Models
# These models define the expected request bodies for creating
# and updating books. They provide automatic validation.
# ------------------------------------------------------------

class BookCreate(BaseModel):
    """Model used when creating a new book."""
    bookId: Optional[str] = None
    title: str
    series: Optional[str] = None
    author: Optional[str] = None
    rating: Optional[float] = None
    description: Optional[str] = None
    language: Optional[str] = None
    isbn: Optional[str] = None
    genres: Optional[str] = None
    characters: Optional[str] = None
    bookFormat: Optional[str] = None
    edition: Optional[str] = None
    pages: Optional[int] = None
    publisher: Optional[str] = None
    publishDate: Optional[str] = None
    firstPublishDate: Optional[str] = None
    awards: Optional[str] = None
```

```
    numRatings: Optional[int] = None
    ratingsByStars: Optional[str] = None
    likedPercent: Optional[float] = None
    setting: Optional[str] = None
    coverImg: Optional[str] = None
    bbeScore: Optional[float] = None
    bbeVotes: Optional[int] = None
    price: Optional[float] = None


class BookUpdate(BaseModel):
    """Model used when updating an existing book."""
    bookId: Optional[str] = None
    title: Optional[str] = None
    series: Optional[str] = None
    author: Optional[str] = None
    rating: Optional[float] = None
    description: Optional[str] = None
    language: Optional[str] = None
    isbn: Optional[str] = None
    genres: Optional[str] = None
    characters: Optional[str] = None
    bookFormat: Optional[str] = None
    edition: Optional[str] = None
    pages: Optional[int] = None
    publisher: Optional[str] = None
    publishDate: Optional[str] = None
    firstPublishDate: Optional[str] = None
    awards: Optional[str] = None
    numRatings: Optional[int] = None
    ratingsByStars: Optional[str] = None
    likedPercent: Optional[float] = None
    setting: Optional[str] = None
    coverImg: Optional[str] = None
    bbeScore: Optional[float] = None
    bbeVotes: Optional[int] = None
    price: Optional[float] = None


# ------------------------------------------------------------
# FastAPI application instance
# ------------------------------------------------------------

app = FastAPI()

# SQLite database filename
DB_NAME = "books.db"


# ------------------------------------------------------------
# Database Connection Helper
# ------------------------------------------------------------

def get_connection():
    """
    Create and return a connection to the SQLite database.

    The row_factory option allows rows to be accessed like
    dictionaries, making JSON conversion easier.
    """
    conn = sqlite3.connect(DB_NAME)
    conn.row_factory = sqlite3.Row
    return conn


# ------------------------------------------------------------
# Data Cleaning / Conversion Helpers
```

```
# These functions convert CSV string values into the correct
# Python types before inserting them into the database.
# ------------------------------------------------------------

def to_int(value):
    """Convert a value to an integer if possible."""
    if value is None:
        return None
    value = str(value).strip().replace(",", "")
    if value == "":
        return None
    try:
        return int(float(value))
    except ValueError:
        return None


def to_float(value):
    """Convert a value to a float if possible."""
    if value is None:
        return None
    value = str(value).strip().replace(",", "").replace("£", "").replace("$", "")
    if value == "":
        return None
    try:
        return float(value)
    except ValueError:
        return None


def to_text(value):
    """Convert a value to a clean string."""
    if value is None:
        return None
    value = str(value).strip()
    return value if value != "" else None


# ------------------------------------------------------------
# Database Initialisation
# Creates the books table if it does not already exist.
# ------------------------------------------------------------

def init_db():
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    CREATE TABLE IF NOT EXISTS books (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        bookId TEXT,
        title TEXT NOT NULL,
        series TEXT,
        author TEXT,
        rating REAL,
        description TEXT,
        language TEXT,
        isbn TEXT,
        genres TEXT,
        characters TEXT,
        bookFormat TEXT,
        edition TEXT,
        pages INTEGER,
        publisher TEXT,
        publishDate TEXT,
        firstPublishDate TEXT,
        awards TEXT,
```

```
        numRatings INTEGER,
        ratingsByStars TEXT,
        likedPercent REAL,
        setting TEXT,
        coverImg TEXT,
        bbeScore REAL,
        bbeVotes INTEGER,
        price REAL
    )
    """)

    conn.commit()
    conn.close()


# ------------------------------------------------------------
# CSV Import Function
# Imports the dataset into the database if the table is empty.
# ------------------------------------------------------------

def import_csv():
    if not os.path.exists("books.csv"):
        return

    conn = get_connection()
    cursor = conn.cursor()

    # Check if database already contains data
    cursor.execute("SELECT COUNT(*) FROM books")
    count = cursor.fetchone()[0]

    # Only import CSV if the table is empty
    if count == 0:
        with open("books.csv", newline="", encoding="utf-8") as file:
            reader = csv.DictReader(file)

            for row in reader:
                cursor.execute"""
                INSERT INTO books (
                    bookId, title, series, author, rating, description, language,
                    isbn, genres, characters, bookFormat, edition, pages, publisher,
                    publishDate, firstPublishDate, awards, numRatings, ratingsByStars,
                    likedPercent, setting, coverImg, bbeScore, bbeVotes, price
                )
                VALUES
(?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
                """, (
                    to_text(row.get("bookId")),
                    to_text(row.get("title")),
                    to_text(row.get("series")),
                    to_text(row.get("author")),
                    to_float(row.get("rating")),
                    to_text(row.get("description")),
                    to_text(row.get("language")),
                    to_text(row.get("isbn")),
                    to_text(row.get("genres")),
                    to_text(row.get("characters")),
                    to_text(row.get("bookFormat")),
                    to_text(row.get("edition")),
                    to_int(row.get("pages")),
                    to_text(row.get("publisher")),
                    to_text(row.get("publishDate")),
                    to_text(row.get("firstPublishDate")),
                    to_text(row.get("awards")),
                    to_int(row.get("numRatings")),
                    to_text(row.get("ratingsByStars")),
                    to_float(row.get("likedPercent")),
```

```
                    to_text(row.get("setting")),
                    to_text(row.get("coverImg")),
                    to_float(row.get("bbeScore")),
                    to_int(row.get("bbeVotes")),
                    to_float(row.get("price"))
                ))

    conn.commit()
    conn.close()


# Initialise database and import CSV dataset on startup
init_db()
import_csv()


# ------------------------------------------------------------
# API Endpoints
# ------------------------------------------------------------

@app.get("/")
def root():
    """Simple endpoint to verify that the API is running."""
    return {"message": "Book API running"}


@app.get("/books")
def get_books():
    """Return a list of books (limited to 100 results)."""
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books LIMIT 100")
    books = [dict(row) for row in cursor.fetchall()]

    conn.close()
    return {"books": books}


@app.get("/books/{id}")
def get_book(id: int):
    """Retrieve a single book by its database ID."""
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books WHERE id = ?", (id,))
    book = cursor.fetchone()

    conn.close()

    if book is None:
        raise HTTPException(status_code=404, detail="Book not found")

    return dict(book)


@app.post("/books", status_code=201)
def add_book(book: BookCreate):
    """Create a new book record in the database."""
    conn = get_connection()
    cursor = conn.cursor()

    data = book.model_dump(exclude_none=True)

    columns = ", ".join(data.keys())
    placeholders = ", ".join(["?"] * len(data))
```

```
    cursor.execute(
        f"INSERT INTO books ({columns}) VALUES ({placeholders})",
        tuple(data.values())
    )

    conn.commit()
    new_id = cursor.lastrowid
    conn.close()

    return {"message": "Book added", "id": new_id}


@app.put("/books/{id}")
def update_book(id: int, book: BookUpdate):
    """Update an existing book record."""
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books WHERE id = ?", (id,))
    existing = cursor.fetchone()

    if existing is None:
        conn.close()
        raise HTTPException(status_code=404, detail="Book not found")

    data = book.model_dump(exclude_none=True)

    if not data:
        conn.close()
        raise HTTPException(status_code=400, detail="No fields provided for update")

    updates = ", ".join([f"{key}=?" for key in data.keys()])

    cursor.execute(
        f"UPDATE books SET {updates} WHERE id = ?",
        tuple(data.values()) + (id,)
    )

    conn.commit()
    conn.close()

    return {"message": "Book updated"}


@app.delete("/books/{id}", status_code=204)
def delete_book(id: int):
    """Delete a book record from the database."""
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM books WHERE id = ?", (id,))
    existing = cursor.fetchone()

    if existing is None:
        conn.close()
        raise HTTPException(status_code=404, detail="Book not found")

    cursor.execute("DELETE FROM books WHERE id = ?", (id,))
    conn.commit()
    conn.close()

    return Response(status_code=204)


# ------------------------------------------------------------
# Analytics Endpoints
# ------------------------------------------------------------
```

```python
@app.get("/analytics/genre-trends")
def genre_trends():
    """Return the most common genres in the dataset."""
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
        SELECT genres, COUNT(*) as count
        FROM books
        WHERE genres IS NOT NULL AND genres != ''
        GROUP BY genres
        ORDER BY count DESC
        LIMIT 10
    """)
    results = [dict(row) for row in cursor.fetchall()]
    conn.close()

    return {"genre_trends": results}


@app.get("/analytics/top-rated")
def top_rated():
    """Return the top-rated books."""
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
        SELECT id, title, author, rating
        FROM books
        WHERE rating IS NOT NULL
        ORDER BY rating DESC
        LIMIT 10
    """)
    results = [dict(row) for row in cursor.fetchall()]
    conn.close()

    return {"top_rated": results}


@app.get("/analytics/rating-distribution")
def rating_distribution():
    """Return distribution of books across rating ranges."""
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
        SELECT
            CASE
                WHEN rating < 3 THEN 'Below 3'
                WHEN rating >= 3 AND rating < 4 THEN '3 to 3.9'
                WHEN rating >= 4 AND rating < 4.5 THEN '4 to 4.4'
                ELSE '4.5 and above'
            END AS rating_band,
            COUNT(*) AS count
        FROM books
        WHERE rating IS NOT NULL
        GROUP BY rating_band
        ORDER BY count DESC
    """)
    results = [dict(row) for row in cursor.fetchall()]
    conn.close()

    return {"rating_distribution": results}
```

Why this level of commenting is good for coursework

Markers usually want to see:

- **file overview**
- **section headers**
- **function explanations**
- **brief endpoint descriptions**

This shows you understand the architecture without cluttering the code.

If you want, I can also show you **one tiny documentation trick that lecturers love seeing in FastAPI projects** (it improves your Swagger docs automatically with almost no extra code).